INFORME DE LABORATORIO

(formato estudiante)

INFORMACIÓN BÁSICA					
ASIGNATURA:	INTRODUCCIÓN AL DESARROLLO DE NUEVAS PLATAFORMAS (E)				
TÍTULO DE LA PRÁCTICA:	Listas filtrables con LazyColumn en Jetpack Compose				
NÚMERO DE PRÁCTICA:	8	**AÑO LECTIVO:**	2026-B	**NRO. SEMESTRE:**	VIII
FECHA DE PRESENTACIÓN	24/10/2026	**HORA DE PRESENTACIÓN**	23:59		
INTEGRANTE (s): - Zuñiga Villacorta, Peter Sebastian				**NOTA (0-20)**	
DOCENTE(s): Roxana Evelyn Limache Calatayud					

RESULTADOS Y PRUEBAS

Github Repo: https://github.com/UNSA-IS-2026-B/idnp_lab/tree/main/lab08

1. OBJETIVOS

- Comprender el Composable LazyColumn y su equivalencia técnica con el componente RecyclerView del View System tradicional para la visualización eficiente de datos masivos mediante reciclaje y renderizado bajo demanda (virtualización de UI).
- Implementar una lista estructurada de edificaciones universitarias con al menos tres atributos por elemento (nombre, categoría y descripción) utilizando LazyColumn, Card y componentes tipográficos de Material Design 3.
- Aplicar un campo de entrada de texto interactivo (OutlinedTextField) ligado al estado mutable de la interfaz para ejecutar filtrados en tiempo real sobre la colección de datos durante el ciclo de recomposición.

2. EJERCICIOS RESUELTOS

A continuación se detalla la solución completa del ejercicio guiado correspondiente a la aplicación "ListaFiltrable_Compose", dividida en la estructura de clases del proyecto.

a. Descripción de la solución técnica

El ejercicio resuelto aborda la creación de un componente de interfaz reactiva en Android mediante Jetpack Compose, diseñado bajo el patrón de arquitectura declarativa y el principio de flujo de datos unidireccional (Unidirectional Data Flow - UDF).

La solución resuelve la representación gráfica de un listado de infraestructura universitaria mediante el Composable LazyColumn. A diferencia de un Column convencional con scroll vertical —el cual compone e instancia en memoria la totalidad de los elementos simultáneamente—, LazyColumn implementa virtualización de interfaz de usuario. Esto significa que los componentes visuales solo son creados, medidos y dibujados cuando entran dentro de la ventana gráfica visible (viewport) del dispositivo, ofreciendo una equivalencia técnica directa con el componente RecyclerView del paradigma tradicional (XML / View System), pero prescindiendo de adaptadores, patrones ViewHolder o archivos de diseño independientes.

b. Código Fuente Completo e Integrado

A continuación se presenta el código modularizado por archivos, seguido de un bloque unificado para facilitar su compilación e integración en Android Studio.

Archivo 1: Edificio.kt

Representa el modelo de datos (Domain Entity) inmutable para cada elemento de la lista.

Kotlin

```kotlin
package com.example.listafiltrable_compose

/**
 * Modelo de datos que representa la infraestructura universitaria.
 * @param id Identificador único para el rastreo de clave estable en Compose.
 * @param nombre Denominación oficial del edificio.
 * @param categoria Clasificación funcional (Servicios, Académico, Deportes, Eventos).
 * @param descripcion Detalle de las actividades o funciones que alberga.
 */
data class Edificio(
    val id: Int,
    val nombre: String,
    val categoria: String,
    val descripcion: String
)
```

Archivo 2: FilaEdificio.kt

Composable especializado en la representación visual individual de cada registro utilizando tarjetas de Material Design 3.

Kotlin

```kotlin
package com.example.listafiltrable_compose

import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Card
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
```

```
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp

@Composable
fun FilaEdificio(edificio: Edificio) {
    Card(
        modifier = Modifier
            .fillMaxWidth()
            .padding(vertical = 6.dp)
    ) {
        Column(
            modifier = Modifier.padding(12.dp)
        ) {
            Text(
                text = edificio.nombre,
                fontWeight = FontWeight.Bold,
                fontSize = 18.sp
            )
            Text(
                text = edificio.categoria,
                fontSize = 14.sp,
                color = Color.Gray
            )
            Text(
                text = edificio.descripcion,
                fontSize = 14.sp
            )
        }
    }
}
```

Archivo 3: ListaEdificios.kt
Componente contenedor que administra el estado del filtro de texto, ejecuta la lógica de coincidencia sobre la colección y renderiza la lista virtualizada.

Kotlin

```
package com.example.listafiltrable_compose

import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
```

```
import androidx.compose.material3.OutlinedTextField
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp

@Composable
fun ListaEdificios() {
    // 1. Colección inmutable de datos preservada en memoria durante recomposiciones
    val edificios = remember {
        listOf(
            Edificio(1, "Biblioteca Central", "Servicios", "Préstamo de libros y salas de
estudio"),
            Edificio(2, "Comedor Universitario", "Servicios", "Alimentación para
estudiantes"),
            Edificio(3, "Facultad de Ingeniería", "Académico", "Aulas y laboratorios de
ingeniería"),
            Edificio(4, "Facultad de Ciencias", "Académico", "Aulas y laboratorios de
ciencias"),
            Edificio(5, "Coliseo Universitario", "Deportes", "Actividades deportivas y
eventos"),
            Edificio(6, "Auditorio Central", "Eventos", "Conferencias y ceremonias")
        )
    }

    // 2. Estado reactivo mutable para almacenar el criterio de búsqueda ingresado por el
usuario
    var texto by remember { mutableStateOf("") }

    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(16.dp)
    ) {
        // 3. Campo de entrada interactivo ligado mediante delegación de propiedades (by)
        OutlinedTextField(
            value = texto,
            onValueChange = { texto = it },
            label = { Text("Buscar edificio") },
            modifier = Modifier.fillMaxWidth()
        )

        Spacer(modifier = Modifier.height(12.dp))
```

```
        // 4. Operación de filtrado dinámico en tiempo real (evaluada en cada
recomposición)
        val edificiosFiltrados = edificios.filter {
            it.nombre.contains(texto, ignoreCase = true)
        }

        // 5. Lista perezosa con optimización de claves estables para minimización de
recomposiciones
        LazyColumn(
            modifier = Modifier.fillMaxSize()
        ) {
            items(
                items = edificiosFiltrados,
                key = { it.id } // Clave única por elemento
            ) { edificio ->
                FilaEdificio(edificio = edificio)
            }
        }
    }
}
```

Archivo 4: MainActivity.kt
Punto de entrada de la aplicación que vincula la jerarquía de Composables con la actividad Android.

Kotlin

```
package com.example.listafiltrable_compose

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.material3.Surface
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.compose.ui.tooling.preview.Preview
import com.example.listafiltrable_compose.ui.theme.ListaFiltrable_ComposeTheme

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            ListaFiltrable_ComposeTheme {
                Surface(modifier = Modifier.fillMaxSize()) {
                    ListaEdificios()
                }
            }
        }
    }
```

```
}

@Preview(showBackground = true)
@Composable
fun ListaEdificiosPreview() {
    ListaFiltrable_ComposeTheme {
        ListaEdificios()
    }
}
```

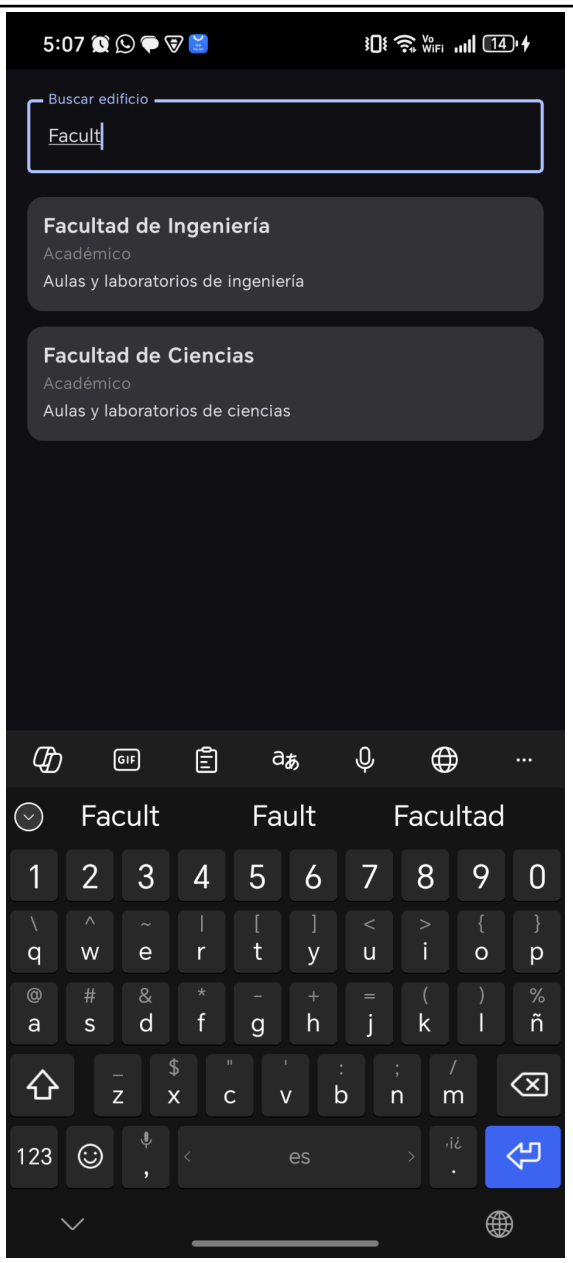
c. Análisis del Flujo de Estado y Ciclo de Recomposición

El funcionamiento dinámico de este ejercicio se basa en cuatro fundamentos de la arquitectura de Jetpack Compose:

- **Preservación del Estado (remember):** La función remember { ... } le indica al runtime de Compose que almacene la instancia de la lista edificios y el valor del string texto en el árbol de composición. Sin remember, la lista de objetos y la variable del campo de texto se reinicializarían en su valor predeterminado cada vez que la pantalla sufra una recomposición.
- **Observabilidad Reactiva (mutableStateOf):** La variable texto utiliza el delegado mutableStateOf(""). Cuando el usuario escribe un carácter en el OutlinedTextField, la función lambda onValueChange se dispara y modifica el valor interno de texto. Al ser un estado observable, Compose detecta la mutación e invalida automáticamente los componentes que leen dicho estado, marcándolos para una recomposición.
- **Filtrado Imperativo en Recomposición (filter):** Durante la recomposición de ListaEdificios, el bloque edificios.filter { it.nombre.contains(texto, ignoreCase = true) } se vuelve a evaluar. Compara la cadena ingresada con el atributo nombre de cada objeto Edificio mediante una búsqueda insensible a mayúsculas o minúsculas (ignoreCase = true), produciendo una nueva lista filtrada edificiosFiltrados en tiempo de ejecución.
- **Identidad Estable (key = { it.id }):** Dentro del bloque LazyColumn, el parámetro key asigna un identificador único basado en it.id. Esto permite que el motor de renderizado de Compose sepa exactamente qué elementos han sido insertados, desplazados o eliminados de la UI. Al cambiar el texto de búsqueda, Jetpack Compose recicla y reordena los elementos existentes sin necesidad de volver a instanciar las tarjetas (Card) completas, optimizando el rendimiento de la CPU y la fluidez del scroll.

d. Ejecución





3. EJERCICIOS PROPUESTOS|

1. Análisis y Solución Técnica por Requerimiento

Para la resolución de los ejercicios propuestos, se amplió el proyecto base mediante la integración de conceptos trabajados en laboratorios previos (persistencia en archivos del Lab 03 y componentes visuales/navegación del Lab 06), orientando la información a la infraestructura real del Campus de Ingenierías de la Universidad Nacional de San Agustín (UNSA) extraída de su plano topográfico.

Requerimiento 1: Carga Dinámica desde Archivo Plano (edificios.txt)

Solución: Se reemplazó la lista estática codificada en duro (hardcoded) por un mecanismo de lectura de archivos de texto almacenados en el directorio privado de la aplicación (Context.MODE_PRIVATE), implementando la función cargarEdificiosDesdeArchivo(context).

Kotlin

```
fun cargarEdificiosDesdeArchivo(context: Context): List<LugarCampus> {
    val fileName = "edificios.txt"
    val file = context.getFileStreamPath(fileName)

    // Crear archivo por defecto si no se encuentra registrado en el almacenamiento
interno
    if (!file.exists()) {
        val contenidoInicial = """
            1|Escuela Profesional de Ingeniería de Sistemas|Aulas / Labs|Laboratorios de
software, inteligencia artificial y redes.|Pabellón especializado equipado con
laboratorios de cómputo de última generación, instalaciones de redes y centros de
investigación de la EPIS.|Computer
            2|Biblioteca de Ingenierías|Académico|Salas de estudio, bibliografía técnica
y bases digitales.|Ofrece amplias salas de lectura individual y grupal, acceso a bases de
datos científicas e información especializada en todas las ramas de la
ingeniería.|MenuBook
            3|Facultad de Arquitectura|Académico|Talleres de diseño, maquetación y
planificación urbana.|Infraestructura con talleres de modelado, diseño bioclimático,
aulas teóricas, laboratorios de dibujo y centros de maquetación.|Architecture
            4|Escuela Profesional de Ingeniería Civil|Aulas / Labs|Laboratorios de
Estructuras, Suelos y Topografía.|Cuenta con laboratorios de mecánica de suelos, ensayo
de materiales de construcción, hidráulica y gabinetes de topografía.|Engineering
            5|Escuela Profesional de Ingeniería Mecánica|Aulas / Labs|Talleres de
manufactura, procesos térmicos y motores.|Instalaciones equipadas con talleres de
soldadura, procesamiento de metales, diseño asistido por computadora (CAD/CAM) y mecánica
pesada.|Build
            6|Ingeniería Química y Metalúrgica|Aulas / Labs|Laboratorios de procesos
químicos, pirometallurgia y análisis.|Pabellones dedicados al estudio de transformaciones
químicas, procesamiento de minerales, balances de materia y ensayos de
materiales.|Science
            7|Facultad de Producción y Servicios|Académico|Decanato, aulas de posgrado y
oficinas administrativas.|Centro administrativo principal de la FIPS que alberga
decanatos, coordinaciones académicas, salas de docentes y auditorios principal y
secundario.|Business
            8|Escuela de Matemática y Física|Académico|Aulas teóricas, laboratorios de
física básica y óptica.|Infraestructura para la enseñanza de ciencias puras, cálculo
applied, física experimental, óptica y laboratorios de investigación teórica.|Functions
            9|Pabellón Nicholson / Pabellón de la Cultura|Cultura|Edificio histórico,
aula magna y eventos culturales.|Pabellón de alto valor arquitectónico patrimonial que
acoge conferencias magistrales, actos protocolarios, congresos y galerías de la comunidad
unsaína.|AccountBalance
            10|Estadio Ho Chi Minh|Deportes|Cancha de fútbol, pista atlética y
tribunas.|Escenario deportivo emblemático del campus para torneos interescuelas,
competencias atléticas universitarias y entrenamiento de selecciones.|SportsSoccer
```

```
            11|Infounsa|Servicios|Centro de informática, redes y soporte
tecnológico.|Oficinas centrales de tecnologías de información, administración de la
infraestructura de red universitaria, servidores y capacitación digital.|Dns
            12|Panificadora e Imprenta Universitaria|Servicios|Unidades de producción de
alimentos e impresiones.|Centros de producción de la universidad encargados del
abastecimiento de panadería para el comedor y servicios de imprenta institucional y
académica.|Store
            13|Ingeniería Geológica|Aulas / Labs|Laboratorio de petrología, mineralogía y
geotecnia.|Laboratorios especializados en el estudio de muestras rocosas, mapas
geológicos, prospección minera y mecánica de rocas.|Terrain
            14|Escuela de Arte|Cultura|Talleres de escultura, pintura y artes
plásticas.|Espacios creativos equipados para el desarrollo de disciplinas artísticas,
exposiciones universitarias y talleres abiertos a la comunidad.|Palette
        """.trimIndent()

        try {
            context.openFileOutput(fileName, Context.MODE_PRIVATE).use { output ->
                output.write(contenidoInicial.toByteArray())
            }
        } catch (e: Exception) {
            e.printStackTrace()
        }
    }

    // Lectura y parseo estricto del archivo
    val lista = mutableListOf<LugarCampus>()
    try {
        context.openFileInput(fileName).bufferedReader().useLines { lines ->
            lines.forEach { line ->
                if (line.isNotBlank()) {
                    val partes = line.split("|")
                    if (partes.size >= 6) {
                        lista.add(
                            LugarCampus(
                                id = partes[0].trim(),
                                nombre = partes[1].trim(),
                                categoria = partes[2].trim(),
                                descripcion = partes[3].trim(),
                                detalleCompleto = partes[4].trim(),
                                icono = obtenerIconoPorClave(partes[5]),
                                colorFondo = obtenerColorPorCategoria(partes[2].trim())
                            )
                        )
                    }
                }
            }
        }
    } catch (e: Exception) {
        e.printStackTrace()
```

```
    }
    return lista
}
```

Mecanismo: La función verifica si el archivo edificios.txt existe localmente mediante context.getFileStreamPath(). Si no existe en la primera ejecución, la aplicación crea el archivo automáticamente insertando 14 registros reales del campus (Sistemas, Civil, Mecánica, Arquitectura, Biblioteca de Ingenierías, Pabellón Nicholson, Estadio Ho Chi Minh, etc.). Los registros se parsean línea por línea utilizando el delimitador pipe (|) con bufferedReader().useLines {}.

Requerimiento 2: Integración de Íconos Vectoriales de Material Design

Solución: En lugar de depender de imágenes en mapa de bits (PNG/JPG) que incrementan el tamaño del APK y requieren la gestión de recursos @drawable, se asoció a cada entidad una clave textual en el archivo plano (Computer, Architecture, Science, Engineering, etc.).

Kotlin

```kotlin
fun obtenerIconoPorClave(clave: String): ImageVector {
    return when (clave.trim()) {
        "Computer" -> Icons.Default.Computer
        "MenuBook" -> Icons.Default.MenuBook
        "Architecture" -> Icons.Default.Architecture
        "Engineering" -> Icons.Default.Engineering
        "Build" -> Icons.Default.Build
        "Science" -> Icons.Default.Science
        "Business" -> Icons.Default.Business
        "Functions" -> Icons.Default.Functions
        "AccountBalance" -> Icons.Default.AccountBalance
        "SportsSoccer" -> Icons.Default.SportsSoccer
        "Dns" -> Icons.Default.Dns
        "Store" -> Icons.Default.Store
        "Terrain" -> Icons.Default.Terrain
        "Palette" -> Icons.Default.Palette
        else -> Icons.Default.Place
    }
}
```

Mecanismo: Se implementó la función obtenerIconoPorClave(clave) que convierte cada identificador de cadena en un ImageVector oficial de la librería androidx.compose.material.icons.filled.*. Esto garantiza escalabilidad, renderizado vectorial nítido a cualquier densidad de pantalla y consumo mínimo de memoria RAM.

Requerimiento 3: Filtrado Multicriterio Reactivo (Texto + Categoría)

Solución: Se diseñó un sistema de filtrado combinado que evalúa en tiempo real dos fuentes de entrada de estado:

- **textoBusqueda:** Estado mutable (mutableStateOf("")) acoplado a un OutlinedTextField que evalúa coincidencias insensibles a mayúsculas/minúsculas sobre los atributos nombre y descripcion.
- **categoriaSeleccionada:** Estado mutable acoplado a un componente interactivo ExposedDropdownMenuBox que permite seleccionar una categoría específica (Aulas / Labs, Académico, Servicios, Deportes, Cultura) o restaurar la vista completa mediante la opción "Todas".

Kotlin

```
@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun EdificiosScreen(
    onLugarClick: (LugarCampus) -> Unit = {}
) {
    val context = LocalContext.current

    // Carga de datos reales desde 'edificios.txt'
    val listaCompleta = remember { cargarEdificiosDesdeArchivo(context) }

    // Estados reactivos de la interfaz
    var textoBusqueda by remember { mutableStateOf("") }
    var categoriaSeleccionada by remember { mutableStateOf("Todas") }
    var menuExpandido by remember { mutableStateOf(false) }

    // Lista de categorías únicas para el menú desplegable
    val categoriasDisponibles = remember(listaCompleta) {
        listOf("Todas") + listaCompleta.map { it.categoria }.distinct()
    }

    // Filtrado multicriterio recalculado eficientemente
    val edificiosFiltrados = remember(textoBusqueda, categoriaSeleccionada,
listaCompleta) {
        listaCompleta.filter { edificio ->
            val coincideTexto = edificio.nombre.contains(textoBusqueda, ignoreCase =
true) ||
                    edificio.descripcion.contains(textoBusqueda, ignoreCase = true)
            val coincideCategoria = categoriaSeleccionada == "Todas" ||
                    edificio.categoria.equals(categoriaSeleccionada, ignoreCase = true)

            coincideTexto && coincideCategoria
        }
    }
    ...
}
```

Optimización: Ambos criterios se evalúan en bloque mediante una expresión relacional lógica AND envuelta dentro de un bloque remember(textoBusqueda, categoriaSeleccionada, listaCompleta), evitando recómputos innecesarios sobre el hilo de interfaz de usuario.

Requerimiento 4: Integración en EdificiosScreen y Renderizado de Tarjetas Personalizadas

Solución: Se reestructuró la pantalla EdificiosScreen reemplazando el diseño genérico por la combinación de tarjetas modulares adaptadas del Laboratorio 06:

- **TarjetaLugar:** Tarjeta amplia con contenedor circular de ícono para categorías de tipo Aulas / Labs.
- **TarjetaCompacta:** Diseño de fila simplificada para la categoría de Servicios.
- **TarjetaEtiquetada:** Estilo visual con insignia flotante superior para las categorías Académico, Deportes y Cultura.

Integración en LazyColumn: La renderización de cada ítem dentro del bloque items() de la LazyColumn utiliza un control de flujo condicional (when (lugar.categoria)), seleccionando dinámicamente el estilo de tarjeta correspondiente para cada elemento.

2. Estructura y Fuente de Datos (edificios.txt)

Estructura de cada registro en el archivo de texto:
id|nombre|categoria|descripcion|detalleCompleto|iconoKey

Textproto

```
1|Escuela Profesional de Ingeniería de Sistemas|Aulas / Labs|Laboratorios de software, inteligencia artificial y redes.|Pabellón especializado equipado con laboratorios de cómputo de última generación, instalaciones de redes y centros de investigación de la EPIS.|Computer
2|Biblioteca de Ingenierías|Académico|Salas de estudio, bibliografía técnica y bases digitales.|Ofrece amplias salas de lectura individual y grupal, acceso a bases de datos científicas e información especializada en todas las ramas de la ingeniería.|MenuBook
3|Facultad de Arquitectura|Académico|Talleres de diseño, maquetación y planificación urbana.|Infraestructura con talleres de modelado, diseño bioclimático, aulas teóricas, laboratorios de dibujo y centros de maquetación.|Architecture
4|Escuela Profesional de Ingeniería Civil|Aulas / Labs|Laboratorios de Estructuras, Suelos y Topografía.|Cuenta con laboratorios de mecánica de suelos, ensayo de materiales de construcción, hidráulica y gabinetes de topografía.|Engineering
5|Escuela Profesional de Ingeniería Mecánica|Aulas / Labs|Talleres de manufactura, procesos térmicos y motores.|Instalaciones equipadas con talleres de soldadura, procesamiento de metales, diseño asistido por computadora (CAD/CAM) y mecánica pesada.|Build
6|Ingeniería Química y Metalúrgica|Aulas / Labs|Laboratorios de procesos químicos, pirometallurgia y análisis.|Pabellones dedicados al estudio de transformaciones químicas, procesamiento de minerales, balances de materia y ensayos de materiales.|Science
7|Facultad de Producción y Servicios|Académico|Decanato, aulas de posgrado y oficinas administrativas.|Centro administrativo principal de la FIPS que alberga decanatos, coordinaciones académicas, salas de docentes y auditorios principal y secundario.|Business
8|Escuela de Matemática y Física|Académico|Aulas teóricas, laboratorios de física básica y óptica.|Infraestructura para la enseñanza de ciencias puras, cálculo aplicado, física experimental, óptica y laboratorios de investigación teórica.|Functions
9|Pabellón Nicholson / Pabellón de la Cultura|Cultura|Edificio histórico, aula magna y eventos culturales.|Pabellón de alto valor arquitectónico patrimonial que acoge
```

```
conferencias magistrales, actos protocolarios, congresos y galerías de la comunidad
unsaína.|AccountBalance
10|Estadio Ho Chi Minh|Deportes|Cancha de fútbol, pista atlética y tribunas.|Escenario
deportivo emblemático del campus para torneos interescuelas, competencias atléticas
universitarias y entrenamiento de selecciones.|SportsSoccer
11|Infounsa|Servicios|Centro de informática, redes y soporte tecnológico.|Oficinas
centrales de tecnologías de información, administración de la infraestructura de red
universitaria, servidores y capacitación digital.|Dns
12|Panificadora e Imprenta Universitaria|Servicios|Unidades de producción de alimentos e
impresiones.|Centros de producción de la universidad encargados del abastecimiento de
panadería para el comedor y servicios de imprenta institucional y académica.|Store
13|Ingeniería Geológica|Aulas / Labs|Laboratorio de petrología, mineralogía y
geotecnia.|Laboratorios especializados en el estudio de muestras rocosas, mapas
geológicos, prospección minera y mecánica de rocas.|Terrain
14|Escuela de Arte|Cultura|Talleres de escultura, pintura y artes plásticas.|Espacios
creativos equipados para el desarrollo de disciplinas artísticas, exposiciones
universitarias y talleres abiertos a la comunidad.|Palette
```

3. Código Fuente Completo del Módulo Propuesto

Carpeta de archivos:
https://github.com/UNSA-IS-2026-B/idnp_lab/tree/main/lab08/ListaFiltrable_Compose/app/src/main/java/com/example/listafiltrable_compose

Archivo 1: Modelos Persistencia.kt
Archivo 2: TarjetasCampus.kt
Archivo 3: EdificiosScreen.kt
Archivo 4: MainActivity.kt **(Punto de Entrada)**

4. Guía de Ejecución y Pruebas de Funcionamiento

a. Requisitos Previos de Compilación

Para evitar errores de referencias no resueltas en los íconos de Material Design (Unresolved reference 'Icons'), se incluye la siguiente dependencia en el archivo build.gradle.kts (Module :app):

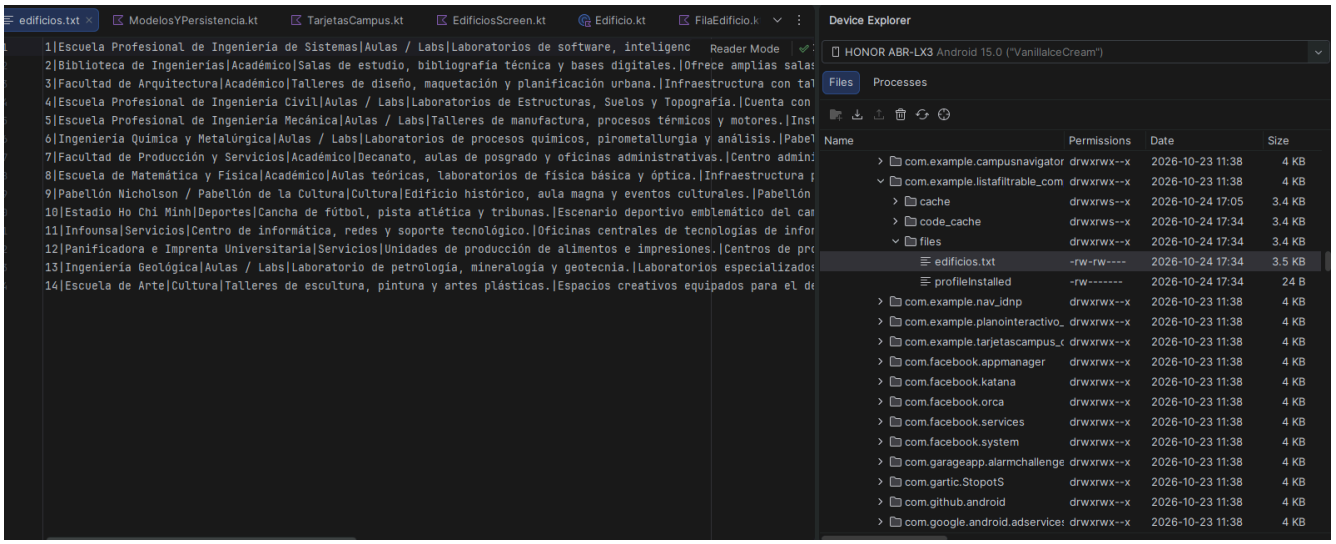
```kotlin
dependencies {
    implementation("androidx.compose.material:material-icons-extended")
}
```

b. Fases de Ejecución en el Emulador

1. Inicialización y Generación del Archivo de Persistencia:

Al presionar el botón Run 'app' (Shift + F10) en Android Studio, la aplicación invoca la función cargarEdificiosDesdeArchivo(context). Como el archivo edificios.txt no existe en la primera ejecución dentro del directorio /data/data/com.example.listafiltrable_compose/files/, se invoca openFileOutput, escribiendo las 14

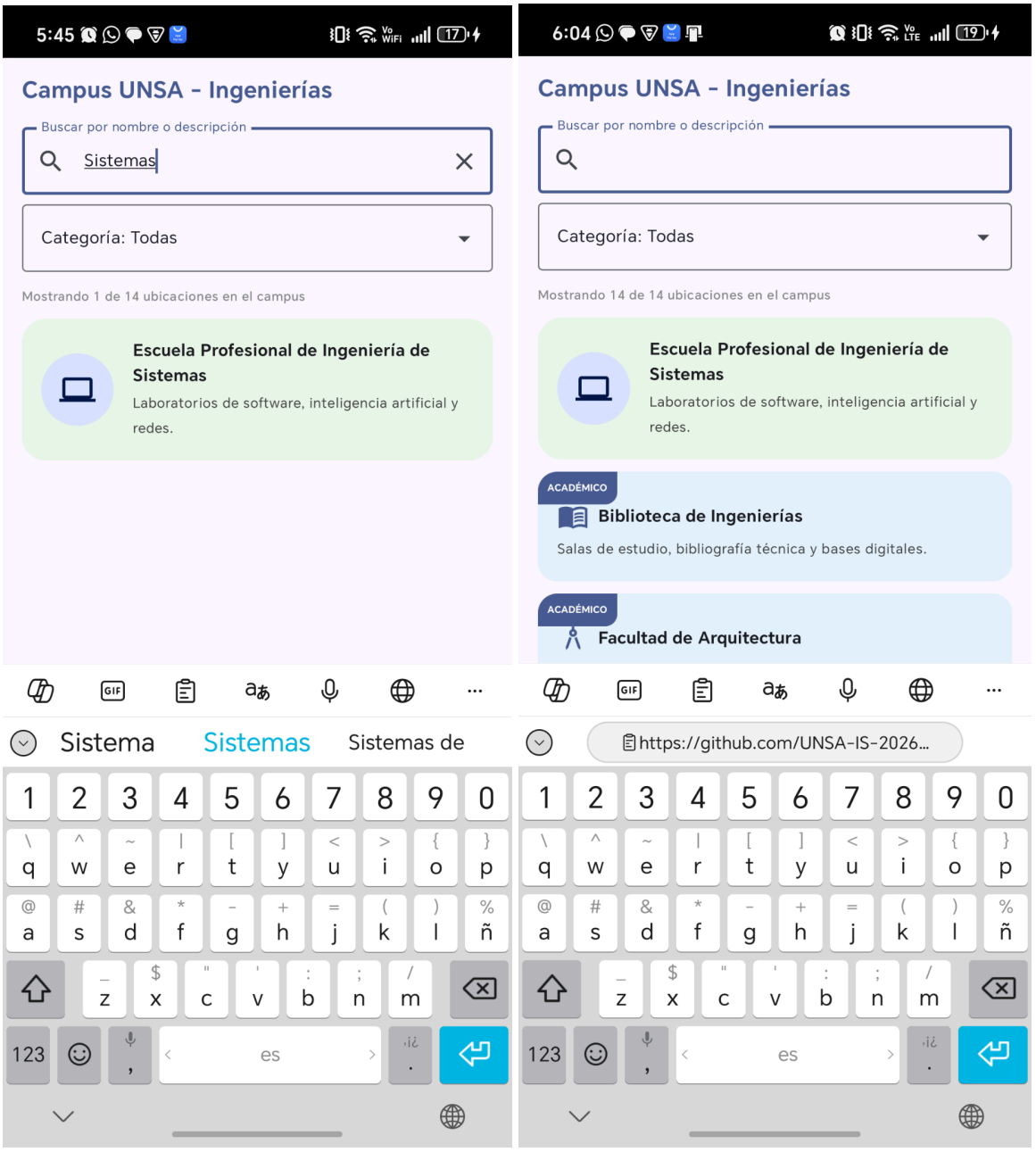
ubicaciones del campus. Inmediatamente después, el archivo se lee y se renderizan los 14 componentes iniciales dentro de la LazyColumn.



2. Prueba de Filtrado por Cadena de Texto:
Acción: El usuario ingresa la palabra "Sistemas" en el campo OutlinedTextField.

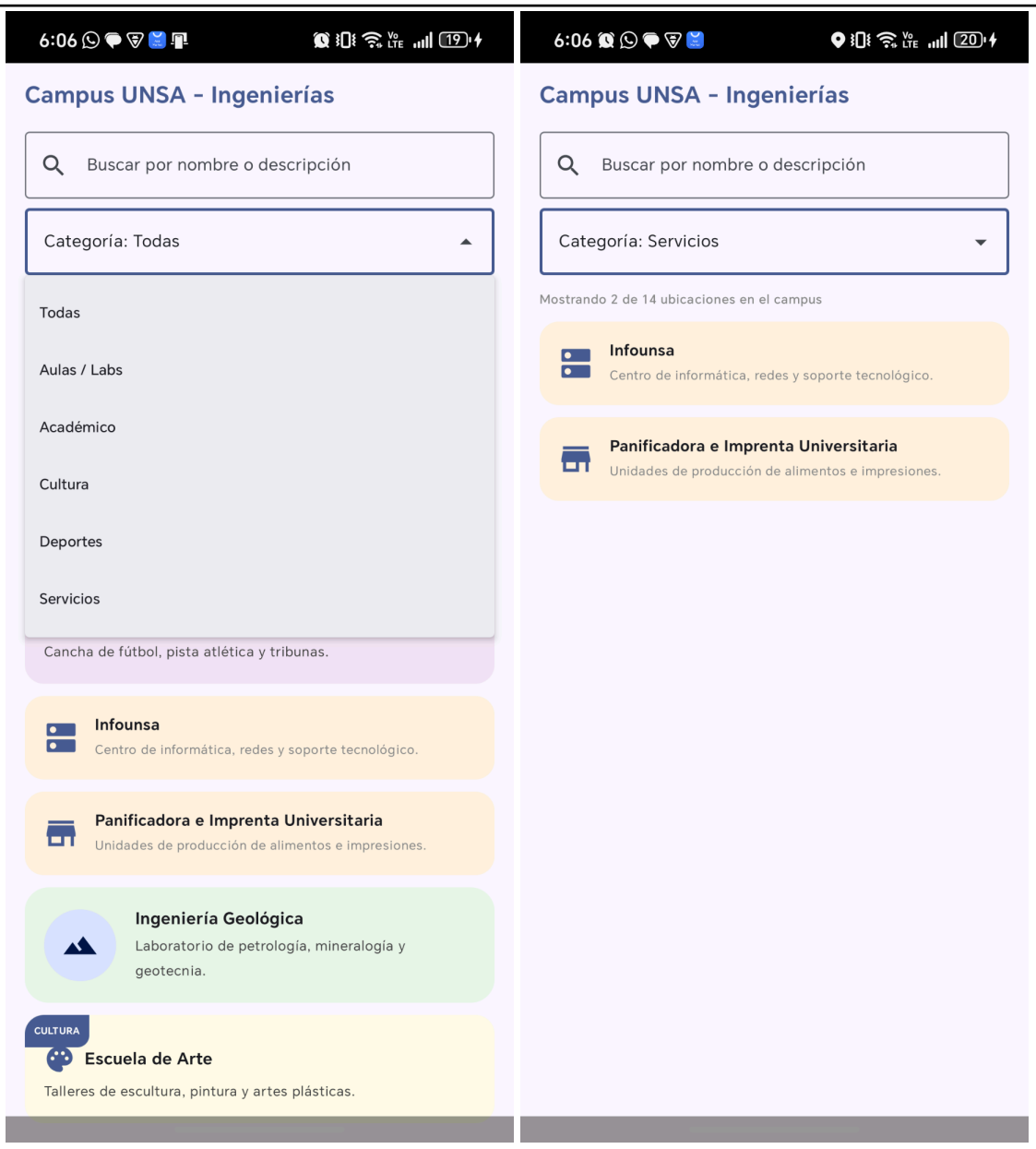
Resultado de UI: La recomposición actualiza la lista mostrando únicamente la tarjeta de la Escuela Profesional de Ingeniería de Sistemas. El contador superior se actualiza dinámicamente a "Mostrando 1 de 14 ubicaciones en el campus".

Limpieza: Al pulsar sobre el ícono Clear (X), el texto se vacía y la lista restaura los 14 elementos de manera instantánea.



3. Prueba de Filtrado por Categoría (Dropdown):
Acción: Se hace clic sobre el menú desplegable ExposedDropdownMenuBox y se selecciona la opción "Servicios".

Resultado de UI: La LazyColumn filtra y renderiza en formato TarjetaCompacta los tres edificios correspondientes a dicha categoría: Infounsa, Panificadora e Imprenta Universitaria, y el Comedor Universitario.



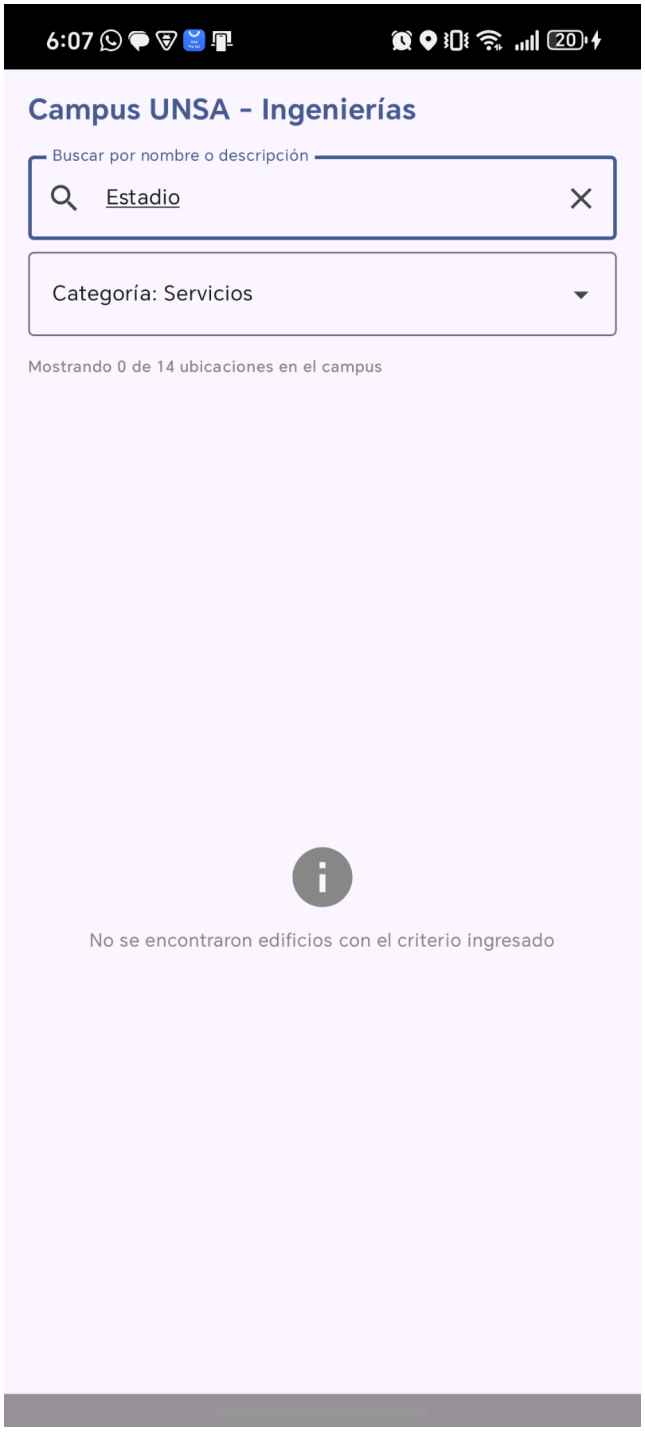
4. Prueba Combinada y Estado Vacío (Empty State):
Acción: Se mantiene seleccionada la categoría "Servicios" y se ingresa en la caja de búsqueda el texto "Estadio".

Resultado de UI: Al no existir coincidencia mutua que satisfaga ambas condiciones relacionales, el renderizado condicional oculta la LazyColumn y despliega en el centro de la pantalla el ícono de información (Icons.Default.Info) junto con el mensaje de advertencia "No se encontraron edificios con el criterio ingresado".

5. Prueba de Evento Interactivo (Toast):
Acción: El usuario selecciona la tarjeta correspondiente a la Facultad de Arquitectura.

Resultado de UI: El evento onClick ejecuta la función lambda que emite una notificación flotante Toast confirmando la selección interactiva del elemento.

4.



5.

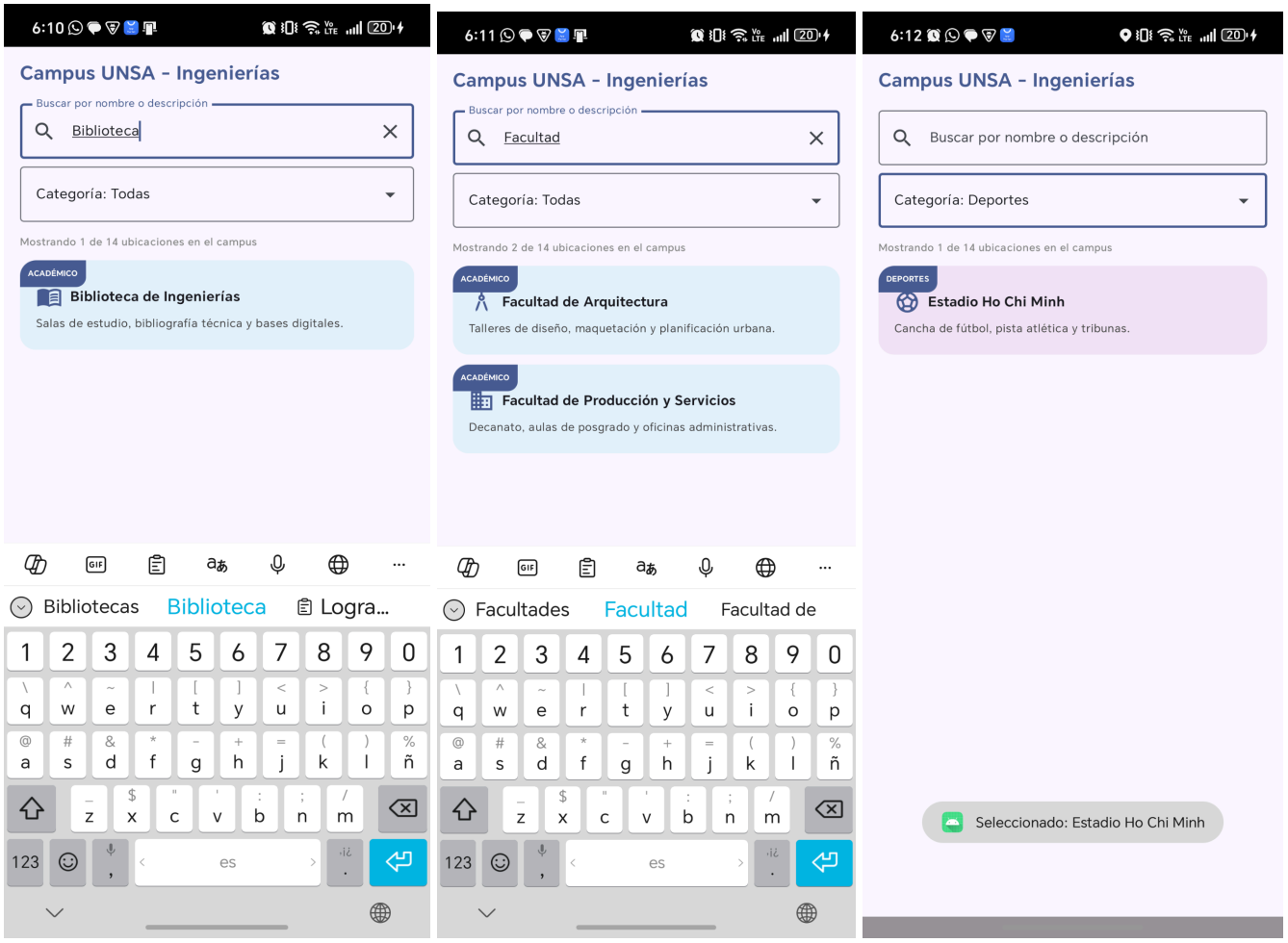


CUESTIONARIO:

1. **Lograste resolver completamente el Ejercicio/Problema Resuelto por el Docente (lista de edificaciones que se filtra en tiempo real al escribir en el campo de búsqueda)? Adjunta una captura de pantalla del emulador mostrando el resultado como evidencia.**

Sí, el ejercicio resuelto fue implementado y verificado satisfactoriamente en el entorno de desarrollo Android Studio utilizando un teléfono móvil personal. La aplicación ejecuta el ciclo de vida reactivo de forma correcta: al ingresar cualquier carácter en el componente OutlinedTextField, la lambda onValueChange actualiza el estado texto, desencadenando la recomposición de la pantalla y actualizando la LazyColumn de forma instantánea.

Asimismo, se verificó el comportamiento ante búsquedas sin coincidencias (comprobando que la pantalla no arroja excepciones de índice ni crashes, mostrando la lista vacía) y ante la limpieza del campo de búsqueda (restaurando la totalidad de los registros de forma inmediata).



2. **En el video de referencia (https://www.youtube.com/watch?v=C4EkfVFC0cg) se aborda el uso de LazyColumn. ¿Qué términos o conceptos nuevos, o que amplían lo visto en la sesión de teoría, se mencionan en el video? Explica brevemente.**

El recurso complementa la teoría mediante los siguientes conceptos de la arquitectura de Jetpack Compose:

- **Scope de Lista Lazy (LazyListScope):** Se introduce el bloque DSL que define el contenido de la lista perezosa. A diferencia de un contenedor tradicional como Column que acepta Composables directamente en su cuerpo, LazyColumn expone un LazyListScope donde se utilizan funciones de extensión específicas como item {} (para elementos individuales como encabezados) e items() (para colecciones de datos).
- **Parámetros de Maquetación DSL (contentPadding y verticalArrangement):** Se profundiza en el uso de PaddingValues dentro del parámetro contentPadding para agregar márgenes internos alrededor de toda la lista navegable sin recortar las sombras ni el efecto de desplazamiento (overscroll), y en Arrangement.spacedBy(), que distribuye automáticamente la separación en píxeles independientes de densidad (dp) entre tarjeta y tarjeta sin necesidad de instanciar componentes Spacer individuales dentro de cada ítem.
- **Elevación de Estado (State Hoisting) en Listas:** Se demuestra cómo la delegación de estados mediante remember y mutableStateOf permite desacoplar los eventos de entrada del usuario del renderizado gráfico de las filas, garantizando que los cambios de estado no destruyan el desplazamiento de la pantalla.

3. **¿Por qué es recomendable pasar un parámetro key = { it.id } a la función items() de una LazyColumn? ¿Qué podría ocurrir si no se especifica al filtrar o reordenar la lista?**

El parámetro key asigna una identidad única, estable e inmutable a cada elemento dentro de la colección que renderiza la LazyColumn.

Importancia Técnica:

Por defecto, si no se proporciona una key explícita, Jetpack Compose asigna identificadores implícitos basados estrictamente en la posición ordinal (índice 0, 1, 2...) del elemento dentro de la lista. Cuando la colección es inmutable o no cambia de orden, este comportamiento por defecto es suficiente.

Consecuencias de Omitir key al Filtrar o Reordenar:

- **Pérdida de Estado Interno:** Si un ítem contiene un estado local propio (por ejemplo, un checkbox marcado, un texto expandido, un campo editado o una animación en curso) y la lista se filtra o reordena, el elemento que pasa de la posición 3 a la posición 0 heredará erróneamente el estado visual que antes pertenecía al elemento ubicado originalmente en la posición 0.
- **Degradación del Rendimiento (Recomposición Innecesaria):** Cuando la lista cambia (por ejemplo, al escribir en el buscador), Compose no puede determinar si un objeto fue movido o eliminado. Al cambiar las posiciones relativas, el motor invalida y vuelve a componer (destruye y redibuja) todos los nodos de la UI visibles desde cero, anulando la optimización del reciclaje de vistas y provocando tirones gráficos (jank) en listas extensas.

CONCLUSIONES

1. Se comprendió y validó el funcionamiento del Composable LazyColumn como la solución estándar en Jetpack Compose para la virtualización de interfaces de usuario. Se demostró que su arquitectura renderiza exclusivamente los elementos dentro del área visible del dispositivo, ofreciendo una equivalencia técnica superior a RecyclerView al eliminar la necesidad de configurar archivos XML, adaptadores (Adapter) y contenedores de vistas (ViewHolder), reduciendo significativamente la complejidad del código fuente.

2. Se implementó de manera exitosa la representación gráfica estructurada de los edificios del campus universitario, integrando modelos de datos inmutables con atributos completos (nombre, categoría, descripción e ícono vectorial). Mediante el uso de tarjetas de Material Design 3 (Card, Surface) y componentes tipográficos, se logró una interfaz limpia, legible y modularizada en archivos de Kotlin independientes.

3. Se aplicó eficazmente la gestión de estados reactivos utilizando remember, mutableStateOf y el patrón de filtrado continuo en el ciclo de recomposición. La vinculación del campo OutlinedTextField con la colección de datos garantizó un filtrado en tiempo real ágil y dinámico, mientras que la optimización computacional con remember(llaves) y la definición de claves únicas (key = { it.id }) aseguraron la fluidez de la interfaz ante cualquier interacción de búsqueda.

REFERENCIAS Y BIBLIOGRAFÍA

[1] Android Developers, "Lists and grids". [En línea]. Disponible: https://developer.android.com/develop/ui/compose/lists

[2] "LazyColumn en Android Jetpack Compose. Lista con texto y botones", video. [En línea]. Disponible: LazyColumn en Android Jetpack Compose. Lista con texto y botones